1. Unity Catalog Table

A Unity Catalog table is a table registered and governed through Unity Catalog.

Example:

datax_catalog.bronze.orders

It follows the three-level namespace:

catalog.schema.table

Unity Catalog manages the table's:

- Metadata
- Permissions
- Governance

Delta Lake provides transactional capabilities such as:

- `UPDATE`
- `DELETE`
- `MERGE`

Interview Point

Unity Catalog → Governance & Access
Delta Lake → Table Transactions

They work together but solve different problems.

---

2. Creating a Managed Delta Table

The class reads `orders.csv` from the existing Unity Catalog Volume and writes it as:

```
(
   orders_df.write
   .format("delta")
   .mode("overwrite")
   .saveAsTable("datax_catalog.bronze.orders")
)
```

Because no external location is specified, the table uses configured Unity Catalog managed storage.

After creation:

```
SELECT *
FROM datax_catalog.bronze.orders;
```

Production Point

`overwrite` is used here for a controlled demo.

In production, be careful because it can replace existing data. Depending on the requirement, incremental pipelines may use:

APPEND
MERGE

instead.

---

3. Inspecting a Table

Two useful commands:

```
DESCRIBE TABLE datax_catalog.bronze.orders;
```

Useful for understanding:

- Columns
- Data types

For detailed Delta information:

```
DESCRIBE DETAIL datax_catalog.bronze.orders;
```

Useful for information such as:

- Format
- Storage location
- Table metadata

As a Data Engineer, always understand an existing table before modifying it.

---

4. What Is a View?

A **view is a saved SQL query** that presents data from one or more underlying objects.

Example:

datax_catalog.gold.completed_orders_vw

The class creates a view containing only completed orders:

```
CREATE OR REPLACE VIEW
datax_catalog.gold.completed_orders_vw
AS

SELECT
    order_id,
    customer_id,
    product_id,
    order_date,
    quantity,
    unit_price,
    quantity * unit_price AS order_amount

FROM datax_catalog.bronze.orders

WHERE order_status = 'COMPLETED';
```

---

5. Table vs View

Table	Standard View
Stores data	Stores a SQL query definition
Contains persisted records	Reads underlying objects when queried
Can support Delta operations	Mainly provides abstraction and reusable logic
Example: `bronze.orders`	Example: `gold.completed_orders_vw`

Interview Answer

A table stores data, whereas a standard view stores query logic and retrieves data from its underlying objects when queried.

---

6. Why Do We Use Views?

Views can be useful for:

- Reusable business logic
- Hiding unnecessary columns or rows
- Providing simpler access to consumers
- Controlled data exposure

Example:

Bronze Orders
↓
Completed Orders View
↓
Reporting Team

Instead of exposing all raw order records, users can query only the required view.

Important

A standard view does **not automatically improve performance**.

It does not store a separate copy of the result in this example.

---

7. Important Unity Catalog Permissions

You should remember these privileges.

`USE CATALOG`

Allows use of the catalog when accessing objects inside it.

`USE SCHEMA`

Allows use of a schema.

`SELECT`

Allows reading table or view data.

`MODIFY`

Allows supported changes to table data.

`CREATE TABLE`

Allows creation of tables inside a schema.

For many table queries, the permission chain is conceptually:

```
USE CATALOG
   ↓
USE SCHEMA
   ↓
SELECT
```

---

8. SHOW GRANTS

To inspect permissions:

```
SHOW GRANTS ON CATALOG datax_catalog;
SHOW GRANTS ON SCHEMA datax_catalog.bronze;
SHOW GRANTS ON TABLE datax_catalog.bronze.orders;
```

This is especially useful while troubleshooting permission errors.

---

9. GRANT

Example: give an analyst group access to a Gold view.

```
GRANT USE CATALOG
ON CATALOG datax_catalog
TO `datax_analysts`;
GRANT USE SCHEMA
ON SCHEMA datax_catalog.gold
```

TO `datax_analysts`;
GRANT SELECT
ON VIEW datax_catalog.gold.completed_orders_vw
TO `datax_analysts`;

Notice that the analysts receive read access to the required view.

They are **not** given unnecessary `MODIFY` access to the Bronze table.

---

10. Least Privilege

Least privilege means giving users only the permissions required to perform their job.

Example:

Analyst
→ SELECT on Gold view

Data Pipeline
→ Required modification privileges

Reporting User
→ No unnecessary access to Bronze

This reduces the risk of accidental changes.

---

11. REVOKE

Remove a privilege using:

REVOKE SELECT
ON VIEW datax_catalog.gold.completed_orders_vw
FROM `datax_analysts`;

But remember:

REVOKE one permission
≠
User definitely loses all access

A user may still get access through:

- Another group
- Inherited privileges
- Another applicable permission

So always check **effective permissions**.

---

12. Production Scenario — Reporting Access

Suppose the finance team only needs completed-order revenue.

A better approach is:

Bronze Data
↓
Validation / Transformation
↓
Gold View or Reporting Table
↓
Finance Team
↓
SELECT Access

Avoid unnecessarily exposing raw or intermediate tables to reporting users.

Groups are also preferred over individual-user grants because they are easier to manage as users join or leave teams.

---

13. Permission-Denied Troubleshooting

Scenario

An analyst says:

> I can see the table in Catalog Explorer, but SELECT gives permission denied.

Check in this order:

User Identity
↓
Group Membership
↓
USE CATALOG
↓
USE SCHEMA
↓
SELECT on Table/View
↓
Inherited Permissions
↓
Workspace Restrictions

For views, also check:

- View ownership
- Underlying dependencies
- Applicable view security requirements

Important Interview Point

Do **not** immediately give:

ALL PRIVILEGES

Identify the missing permission and fix only that.

---

14. Common Mistakes

1. Using `overwrite` carelessly

It can replace important existing table data.

2. Thinking a standard view stores another copy of data

It stores the query definition.

3. Assuming a view automatically improves performance

Not necessarily.

4. Giving analysts `MODIFY`

If they only need reporting access, `SELECT` may be enough.

5. Forgetting `USE CATALOG` and `USE SCHEMA`

Object-level permissions alone may not be sufficient.

6. Giving permissions individually everywhere

Prefer groups for easier access management.

7. Exposing raw Bronze data directly for reporting

Use appropriately validated and transformed datasets when required.

---

15. Interview Questions

Q1. What is a Unity Catalog-managed table?

A table whose metadata, managed storage lifecycle, and access are governed through Unity Catalog.

Q2. What is the difference between a table and a view?

A table stores data.

A standard view stores a SQL query definition and reads its underlying objects when queried.

Q3. Does a standard view automatically improve performance?

No. It does not automatically precompute or persist its result.

Q4. What permissions are generally required to query a Unity Catalog table?

Typically:

USE CATALOG
USE SCHEMA
SELECT

subject to applicable access rules.

Q5. What is the difference between `SELECT` and `MODIFY`?

SELECT → Read data
MODIFY → Perform supported data changes

Q6. What is least-privilege access?

Providing only the minimum permissions required for a user or workload to perform its job.

Q7. Why are groups preferred for permissions?

Groups make access easier to manage when people join, leave, or change responsibilities.

Q8. Can a user query a standard Unity Catalog view without direct access to its underlying table?

Yes, provided the required privileges on the view and containing objects exist and the view's security requirements are satisfied.

Q9. How would you troubleshoot a permission error?

Check:

Identity
→ Group Membership
→ USE CATALOG

→ USE SCHEMA
→ Object Permission
→ Inherited Access
→ Other Restrictions

---

16. Scenario-Based Interview Questions

Scenario 1

Finance only needs completed orders. Would you give them access to the Bronze table?

Not necessarily.

Expose an appropriate Gold view or reporting object and provide only the required read access.

Scenario 2

A user has `SELECT` but still cannot query the table. What could be missing?

Check whether they also have the required:

USE CATALOG
USE SCHEMA

and whether other restrictions apply.

Scenario 3

An analyst requests `ALL PRIVILEGES` because their query fails. What would you do?

Troubleshoot the exact missing permission instead of granting broad access.

Scenario 4

You revoke SELECT from a group, but the user can still query the object. Why?

The user may receive access from another group or inherited privilege.

---

17. Commands to Remember

```
-- Table metadata
DESCRIBE TABLE datax_catalog.bronze.orders;

-- Delta details
DESCRIBE DETAIL datax_catalog.bronze.orders;

-- Permissions
SHOW GRANTS ON TABLE datax_catalog.bronze.orders;

-- Grant read access
GRANT SELECT
ON VIEW datax_catalog.gold.completed_orders_vw
TO `datax_analysts`;

-- Remove grant
REVOKE SELECT
ON VIEW datax_catalog.gold.completed_orders_vw
FROM `datax_analysts`;
```

Quick Revision

Unity Catalog Table
→ Governed structured data

View
→ Saved reusable SQL query

USE CATALOG
→ Catalog access

USE SCHEMA
→ Schema access

SELECT
→ Read

MODIFY

→ Change table data

GRANT
→ Give privilege

REVOKE
→ Remove privilege

SHOW GRANTS
→ Inspect privileges

Least Privilege
→ Give only required access

Final Takeaway

Unity Catalog tables provide governed structured data, views provide reusable and controlled access to that data, and Unity Catalog permissions control who can read or modify those objects.